

Project Title: POCKETSMART AI

Your Smart Budget & Recommendation Assistant

College Name: Government Arts and Science College, Manur.

College Code: MSU142

Department: B.SC Computer Science

Team Id: 6ab21524d329d50f0db6120b

Team Leader: Esakki Aruna S NM ID: 55055706AAD495BADB04C8C4D3B478B8

Team Member: Utchi Makali NM ID: 0BBEDB52C642EC0B0BA4CB4B22DE9E2D

Team Member: Kombu Madathi NM ID: 86EE18ABFB36AEF83784FA01234C5948

Team Member: S Ulagammal NM ID: B59DED29E07C4F78BCF442535560F984

1.PROJECT OVERVIEW

PocketSmart AI is a personal budgeting and recommendation assistant that helps users record income and expenses, create budgets, analyze spending patterns, and receive simple personalized recommendations. The project should combine a userfriendly interface, a structured database, budget calculations, reports, and an AI/data-driven recommendation component.

2. PROJECT PURPOSE

- ✚ Make personal expense tracking simple and organized.
- ✚ Help users compare actual spending with planned budgets.
- ✚ Provide category-wise and time-based spending summaries.
- ✚ Identify spending patterns from recorded transactions.
- ✚ Generate understandable recommendations based on available data.
- ✚ Demonstrate practical use of database, software engineering, analytics, and AI concepts.

3. MAIN PROJECT OBJECTIVES

- ✚ Develop a secure user account and authentication mechanism.
- ✚ Allow users to record income and expenses.
- ✚ Allow transactions to be categorized.
- ✚ Allow users to create monthly or category-based budgets.
- ✚ Calculate total income, total expenses, and remaining balance.
- ✚ Display useful dashboards and reports.
- ✚ Detect budget warnings and spending trends.
- ✚ Generate personalized recommendations.
- ✚ Store data reliably in a database.
- ✚ Provide validation, error handling, and basic security.

4. FUNCTIONAL REQUIREMENTS

Requirement	Description
FR-01: User Registration	The system shall allow a new user to create an account using required profile information.
FR-02: User Login	The system shall authenticate registered users and provide access only to their own financial records.
FR-03: Dashboard	The system shall display current balance, income, expenses, budget status, and recent transactions.
FR-04: Add Income	The user shall be able to enter income amount, date, source/category, payment method, and optional notes.
FR-05: Add Expense	The user shall be able to enter expense amount, date, category, payment method, and optional notes.
FR-06: Edit/Delete Transaction	The user shall be able to modify or remove an existing transaction.

FR-07: Categories	The system shall provide default categories and optionally allow custom categories.
FR-08: Budget Creation	The user shall be able to define a budget for a month and/or individual category.
FR-09: Budget Monitoring	The system shall compare actual
	spending with budget limits and show remaining or exceeded amounts.
FR-10: Reports	The system shall provide category-wise, monthly, and transaction summaries.
FR-11: Recommendations	The system shall analyze recorded spending and provide relevant budget recommendations.
FR-12: Alerts	The system may notify users when spending approaches or exceeds configured thresholds.
FR-13: Search/Filter	The user shall be able to filter transactions by date, category, type, or amount.
FR-14: Data Persistence	User and transaction information shall be stored in a persistent database.
FR-15: Logout	The system shall provide a secure logout function.

5. NON-FUNCTIONAL REQUIREMENTS

- ✚ Usability: The interface should be simple enough for a first-time user.
- ✚ Performance: Dashboard and common database operations should respond quickly under normal usage.
- ✚ Reliability: Valid transactions should be stored consistently without accidental data loss.
- ✚ Security: Authentication, authorization, input validation, and password hashing should be implemented.

- ✚ Privacy: Users should only be able to view and modify their own financial information.
- ✚ Maintainability: Code should be modular and documented so features can be updated easily.
- ✚ Scalability: The design should allow additional users, categories, and recommendation rules.
- ✚ Compatibility: The application should run in the selected target environment and common screen sizes.

6. RECOMMENDED MODULE STRUCTURE

Module 1 – Authentication

Registration, login, logout, password handling, and user session management.

Module 2 – User Profile

Basic profile settings and preferences.

Module 3 – Transaction Management

Add, view, edit, delete, search, and filter income/expense records.

Module 4 – Category Management

Default and custom transaction categories.

Module 5 – Budget Management

Create, update, monitor, and summarize budgets.

Module 6 – Dashboard & Reports

Balance, totals, charts, category summaries, and monthly reports.

Module 7 – AI Recommendation

Analyze spending patterns and generate recommendations.

Module 8 – Notifications

Budget warnings and optional recommendation alerts.

Module 9 – Database

Persistent storage and relationships between users, transactions, budgets, and recommendations.

7. RECOMMENDED TECHNOLOGY STACK

Layer	Recommended Technology
Frontend	HTML5, CSS3, JavaScript; or a suitable framework such as React if required.
Backend	Python with Flask/Django, or another suitable backend framework.
Database	SQLite for a simple academic prototype; MySQL/PostgreSQL/Firebase for an expanded implementation.
AI/Data Analysis	Python, Pandas, NumPy, and optional
	scikit-learn for machine-learning experiments.
Charts	Chart.js, Matplotlib, or another suitable visualization library.
Development Tool	Visual Studio Code or equivalent IDE.
Version Control	Git/GitHub may be used for source-code management.

8. HARDWARE REQUIREMENTS

- ✦ Processor: Dual-core processor or better.
- ✦ RAM: Minimum 4 GB; 8 GB recommended for development.
- ✦ Storage: At least 500 MB for the application and database, excluding development tools.
- ✦ Display: Standard monitor or mobile-compatible display.
- ✦ Internet: Required only if cloud services, hosted APIs, or online deployment are used.

9. SOFTWARE REQUIREMENTS

- ✦ Operating System: Windows, Linux, or macOS for development.
- ✦ Python 3.x if Python is selected for the backend/AI layer.
- ✦ A database system such as SQLite, MySQL, or PostgreSQL.
- ✦ Web browser such as Chrome, Edge, or Firefox for a web application.
- ✦ IDE/editor such as Visual Studio Code.
- ✦ Required libraries/frameworks listed in the project's dependency file.

10.DATABASE REQUIREMENTS

Suggested entities and important fields:

Entity	Suggested Fields
Users	user_id, name, email, password_hash, created_at
Categories	category_id, user_id, category_name, type
Transactions	transaction_id, user_id, category_id, amount, transaction_type, date,
	payment_method, notes
Budgets	budget_id, user_id, category_id, month, limit_amount
Recommendations	recommendation_id, user_id, category_id, message, created_at, status

Security requirement: passwords must never be stored as plain text. Use a secure password-hashing mechanism.

11. USER WORKFLOW † User

opens the application.

- † User registers or logs in.
- † User enters income and expense transactions.
- † System validates and stores the transaction.
- † Dashboard updates totals and balance.
- † User creates a monthly/category budget.
- † System compares spending with the budget.
- † Analytics component identifies useful spending patterns.
- † Recommendation engine generates suggestions.
- † User views reports and recommendations.
- † User can update transactions or budgets and continue monitoring.

12. AI RECOMMENDATION REQUIREMENTS

The first implementation can use transparent rule-based recommendations. This is easier to test and explain. A machine-learning model may be added later when enough historical data is available.

- ✦ Calculate spending by category and time period.
- ✦ Compare category spending against the corresponding budget.
- ✦ Identify categories that repeatedly exceed a configured threshold.
- ✦ Identify increasing or decreasing spending trends.
- ✦ Generate a short recommendation explaining the reason.
- ✦ Avoid presenting recommendations as guaranteed financial advice.
- ✦ Store recommendation history when required for evaluation.

13. SAMPLE RECOMMENDATION RULES

Rule	Example Logic
Budget warning	If spending reaches 80% of a category budget, show a warning.
Budget exceeded	If spending exceeds 100% of a budget, recommend reviewing that category.
Repeated high spending	If a category exceeds its budget for multiple periods, highlight the recurring pattern.
Trend detection	If spending in a category increases across consecutive periods, show a trend message.
Low remaining balance	If projected/remaining balance becomes low relative to the user's recorded budget, suggest reviewing discretionary expenses.

14. USER INTERFACE REQUIREMENTS †

Login/Register screen.

- † Dashboard screen with summary cards.
- † Add Income screen.
- † Add Expense screen.
- † Transaction history screen.
- † Budget management screen.
- † Category management screen.
- † Reports/Charts screen.
- † AI Recommendations screen.
- † Profile/Settings screen. † Logout option.

15. VALIDATION REQUIREMENTS

- † Amount must be numeric and greater than zero.
- † Required fields must not be empty.
- † Dates must use a valid format.

†

†



Email fields should follow a valid email pattern where applicable.

Budget limits must be valid positive values.

Users must not access another user's records.

Invalid inputs should produce clear error messages.

16. SECURITY REQUIREMENTS

- ✚ Use password hashing rather than storing plain-text passwords.
- ✚ Validate and sanitize user input.
- ✚ Use parameterized database queries or an ORM.
- ✚ Implement session/token protection appropriate to the selected architecture.
- ✚ Restrict database operations by authenticated user ID.
- ✚ Do not expose private financial records in public pages or logs. ✚ Keep API keys and secrets outside source code.

17. TESTING PLAN

Test Case	Input/Action	Expected Result
Login Test	Valid and invalid credentials	Correct authentication behavior
Transaction Test	Add income/expense with valid values	Record appears in history and totals
Validation Test	Empty/invalid amount	Appropriate validation message
Budget Test	Set budget and add expenses	Remaining budget updates correctly
Report Test	Multiple categorized transactions	Correct category/month totals
Recommendation Test	Expense exceeds budget	Expected recommendation is generated
Security Test	Attempt access to another user's record	Access is denied





Logout Test	Click logout	Session is ended and protected
-------------	--------------	--------------------------------

18. DEVELOPMENT PHASES

category modules.

Budget calculation and dashboard.

Reports and charts.

Requirement gathering and project planning.

UI and database design.

 User authentication implementation.

 Transaction and  Recommendation engine.

 Testing and debugging.

 Documentation and final presentation.  Deployment/demo preparation.

19. PROJECT DELIVERABLES

Complete source code.

 Working application/demo.

 Database schema and sample data.

 Project report.

 System architecture/data-flow diagrams.

 Screenshots of major screens.

 Test cases and test results.

 Presentation/PPT for project review.  Installation and user instructions.

20. EXPECTED PROJECT OUTPUT

At the end of the project, a user should be able to register/login, record income and expenses, categorize transactions, set a budget, view spending summaries, monitor budget status, and receive recommendations based on the recorded spending data.

The final application should demonstrate a complete flow from data entry to analysis and recommendation.





21. IMPORTANT IMPLEMENTATION GUIDELINES

- ✦ Keep the interface simple and consistent.
 - Use meaningful variable, function, table, and file names.
 - Separate frontend, backend, database, and recommendation logic where possible.





Validate every important user input.

Test calculations with known sample values before integrating charts.

- ✚ Use sample/demo data during presentation so the recommendation feature can be demonstrated clearly.
- ✚ Document assumptions used by the recommendation engine.
- ✚ Do not claim that the AI provides professional financial advice.
- ✚ Keep the project scope manageable for an academic submission; advanced features can be listed under future enhancements.

22. FINAL CHECKLIST

- ✚ Project title and objectives finalized
- ✚ Requirements documented
- ✚ UI screens designed
- ✚ Database created
- ✚ Authentication completed
- ✚ Income/expense module completed
- ✚ Category module completed
- ✚ Budget module completed
- ✚ Dashboard and reports completed
- ✚ AI recommendation feature completed
- ✚ Validation and security checks completed
- ✚ Test cases executed
- ✚ Screenshots collected
- ✚ Project report completed
- ✚ PPT prepared
- ✚ Final demo tested

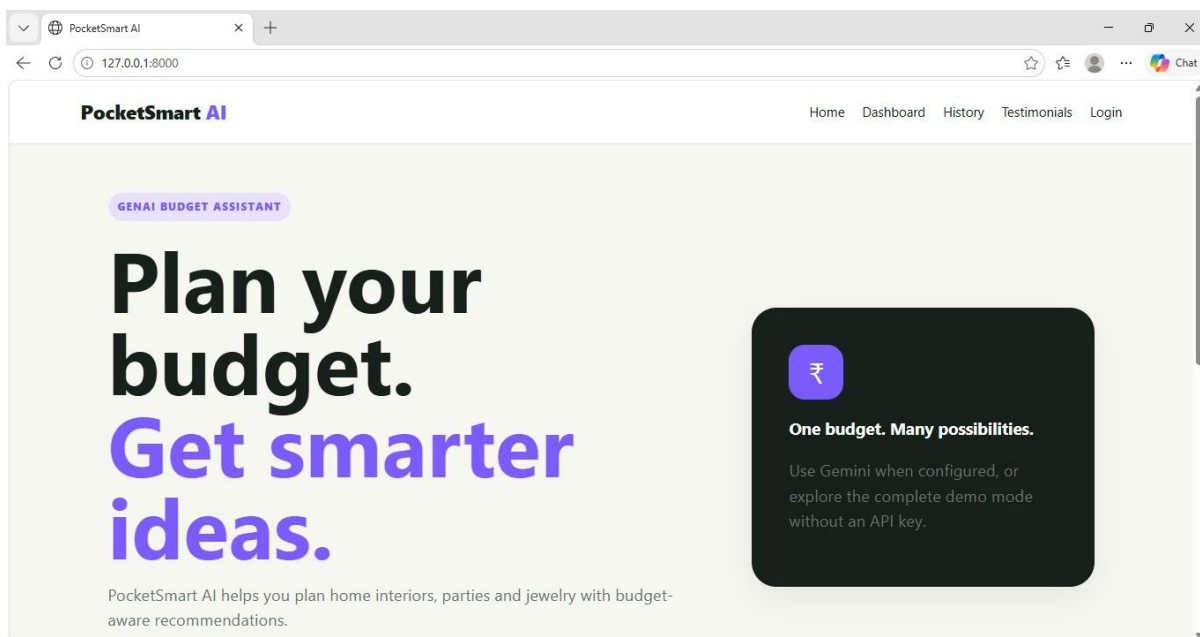
23. NOTE ON FINANCIAL USE

PocketSmart AI is intended as an educational budgeting and planning application. Its calculations and recommendations depend on the data entered by the user. The system should not be presented as a replacement for a qualified financial professional or as a guarantee of financial outcomes.

CONCLUSION

PocketSmart AI: Smart Budget & Recommendation Assistant is designed to help users manage their personal finances in a simple, smart, and efficient way. The system analyzes income, expenses, spending patterns, and user preferences to provide personalized budget recommendations.

24.SCREEN SHOT OUTPUT



Create Account Page:

Register

127.0.0.1:8000/register

PocketSmart AI

Home Dashboard History Testimonials Login

GET STARTED

Create Account

Name

Aruna

Email

esakkiaruna2008@gmail.com

Password

.....

Register

Login Page:

Login

127.0.0.1:8000/login

PocketSmart AI

Home Dashboard History Testimonials Login

WELCOME BACK

Login

Email

esakkiaruna2008@gmail.com

Password

.....

Login

New here? Create an account

Planning Page:

